4. Data Exploration, Standardization, and Normalization

The datasets described in Chapter 3 were written by people, so the texts differ in length and format. Because of this, they contain some problems, such as phone numbers, links, broken characters and Cyrillic letters. These problems need to be fixed, because they make the search results worse. For example, for BM25 a word with one wrong letter is a different word. In addition, phone numbers are personal data and should not be stored. This chapter describes how the data was explored, cleaned and prepared for BM25 and Sentence-BERT. All steps were done in Python with the pandas library [2], and the process and results are saved in the notebook `notebooks/02_data_cleaning.ipynb`. The input files are `candidates_it_v1.csv` (132,220 CVs) and `jobs_it_v1.csv` (20,000 vacancies).

4.1 Data Exploration

Data exploration is the first step before any data changes. It helps to understand how the data looks, find missing values, duplicates and strange records, and decide what needs to be cleaned. In this work the next things were checked: missing values, duplicates, text length, experience level, English level, and problems inside texts. All figures were made with Matplotlib [3].

Missing values and duplicates. The results are shown in Table 4.1. Only three columns have missing values. `Highlights` and `Looking For` are empty for about half of the candidates because these fields are optional on the platform. This is not a problem, because both of them are parts of the `CV` column, and it is never empty. `English Level` is missing for 0.3% of candidates and 5.9% of vacancies. Duplicates were checked in three ways: the same `id`, the same text, and the same text without upper and lower case and extra spaces. No duplicates were found.

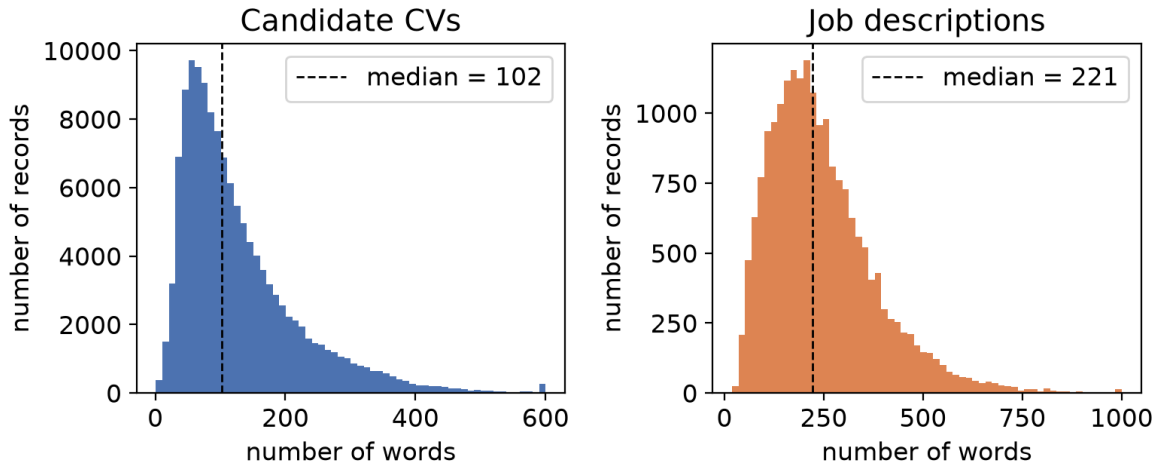
Table 4.1: Missing values and duplicates in the selected data

Check	**Candidates**	**Jobs**
Missing `Highlights`	66,909 (50.6%)	–
Missing `Looking For`	65,387 (49.5%)	–
Missing `English Level`	387 (0.3%)	1,178 (5.9%)
Missing values in other columns	0	0
Same `id`	0	0
Same text	0	0
Same text, ignoring case and spaces	0	0

Text length. Text length was counted in words and Figure 4.1 shows it. The median is 221

words for a vacancy and 102 words for a CV. Some CVs are very short, for example *"Please check my CV file"*. From such texts it is impossible to understand the skills of a candidate, so they cannot be matched with a job.

Figure 4.1: Number of words in CVs and job descriptions (values above 600 and 1,000 words are shown in the last bar)



Experience and English level. Figure 4.2 shows years of experience of candidates, the years of experience required by vacancies, and the English level of both candidates and vacancies. Many candidates are juniors: 30.7% of them have one year of experience or less. At the same time, most of the vacancies ask for 2 to 5 years of experience and only 2.9% of them accept candidates without experience. Also most of the candidates have an intermediate or upper-intermediate English level, and the same two levels are required in 96% of vacancies.

Figure 4.2: Experience of candidates, required experience in vacancies and English level



Problems inside the texts. Texts were checked with regular expressions and the next problems were found during checking:

- **Personal data.** The authors of the dataset tried to anonymize the data [1], no e-mails were found, however, 15 CVs and 4 vacancies still have phone numbers. Also links were found in 1,968 CVs and 652 vacancies. Most of them are links to portfolios and GitHub pages.
- **Special characters.** 1,203 CVs and 498 vacancies have invisible characters, and 1,017 CVs and 653 vacancies have non-breaking spaces. People do not see such things, but for a program it is different information.
- **Cyrillic letters.** All records are marked as English, but 6,455 CVs and 1,528 vacancies have Cyrillic letters. Some of those letters look the same as Latin ones, for example the Cyrillic C (U+0421) and the Latin C (U+0043). People type them by mistake when they switch the language. The word `Creating` looks normal to a person, but for BM25 it is a different word if there is a Cyrillic letter. In some records the Cyrillic letters also make some real words, for example company names or whole sentences.
- **Non-developer roles.** `Primary Keyword` is chosen by people themselves on the platform. Some people have a job title (`Position`) like *3D Artist* or *Game Designer*, but chose `Unity` because they work in it, even though they do not write code. Because the relevance labels are based on `Primary Keyword`, a game designer would get the same label as a Unity developer. Such records can be found by words like artist, animator, designer or illustrator in `Position`.

How these problems were fixed is described in Section 4.2.

4.2 Data Standardization

Data standardization means bringing all texts to one common format. This step has two parts: fixing texts and removing the records that cannot be used.

Text fixes. The texts were fixed with the function `clean_text` (Listing 4.1). No records are removed at this step. The function does the following:

- Broken encoding is repaired with the ftfy library [4].
- Unicode NFKC normalization [5] is applied. It turns special forms of characters into normal ones, for example a non-breaking space becomes a normal space.

- Cyrillic letters that look like Latin ones are replaced with Latin letters. After this, a word with a Cyrillic C becomes a normal English word again.
- Invisible characters are removed.
- Links, e-mails and phone numbers are replaced with `[LINK]`, `[EMAIL]` and `[PHONE]`. Personal data is removed, but it is still clear that a link or a phone number was there.
- Line breaks and spaces are made the same everywhere. For example, several spaces in a row become one space, and more than one empty line becomes one empty line.

These fixes are applied to all text columns.

Listing 4.1: Function for text cleaning

```
def clean_text(text):
   text = ftfy.fix_encoding(text)
   text = unicodedata.normalize("NFKC", text)
   text = fix_similar_letters(text)
   text = re.sub(r"[\u2028\u2029]", "\n", text)
   text = re.sub(INVISIBLE, "", text)
   text = re.sub(LINK, "[LINK]", text)
   text = re.sub(EMAIL, "[EMAIL]", text)
   text = re.sub(PHONE, "[PHONE]", text)
   text = text.replace("\r\n", "\n").replace("\r", "\n")
   text = re.sub(r"[ \t]+", " ", text)
   text = re.sub(r" *\n *", "\n", text)
   text = re.sub(r"\n{3,}", "\n\n", text)
   return text.strip()
```

Removed records. Some records cannot be fixed, so they were removed. Table 4.2 shows how many records were left after each step.

- **Duplicates.** No duplicates were found in Section 4.1, so nothing was removed.
- **Short texts.** CVs and vacancies with less than 20 words were removed: 1,687 CVs and 1 vacancy.
- **Cyrillic text.** After the similar letters were replaced, 3,155 CVs and 616 vacancies still had Cyrillic words. Deleting only these words would break the sentences, for example when a company name is written in Cyrillic. BM25 and Sentence-BERT in this work are used for English texts, so these records were deleted.

- **Non-developer roles.** CVs and vacancies with the words *artist*, *animator*, *designer*, *illustrator* or *3d model* in `Position` were removed: 644 CVs and 77 vacancies.

In total 5,486 CVs (4.1%) and 694 vacancies (3.5%) were removed. 126,734 CVs and 19,306 vacancies were left.

Table 4.2: Number of records after each cleaning step

Step	**CVs**	**Removed**	**Vacancies**	**Removed**
Selected IT data (v1)	132,220	–	20,000	–
Duplicates	132,220	0	20,000	0
Texts shorter than 20 words	130,533	1,687	19,999	1
Texts with Cyrillic words	127,378	3,155	19,383	616
Non-developer roles	126,734	644	19,306	77

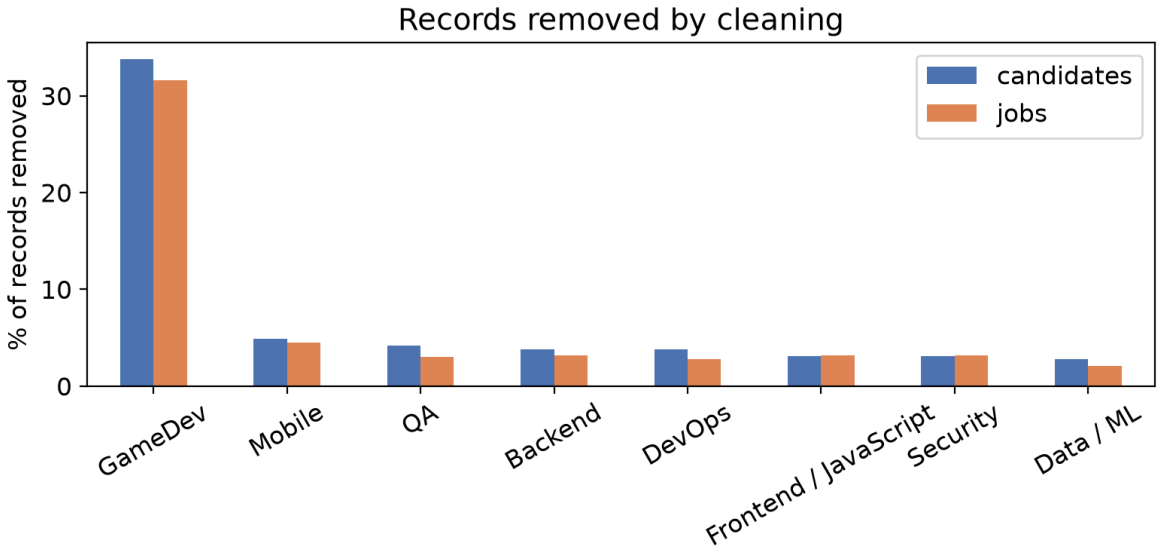
Result of cleaning. Table 4.3 shows the problems from Section 4.1 before and after cleaning. After cleaning all of them are zero.

Table 4.3: Number of texts with problems before and after cleaning

	CVs		**Vacancies**	
Problem	**Before**	**After**	**Before**	**After**
E-mail	0	0	0	0
Link	1,968	0	652	0
Phone number	15	0	4	0
Invisible characters	1,203	0	498	0
Non-breaking spaces	1,017	0	653	0
Broken encoding	0	0	1	0
Latin and Cyrillic in one word	2,744	0	813	0
Cyrillic letters	6,455	0	1,528	0

Figure 4.3 shows what part of each role family was removed. In most families it is from 2% to 5%. GameDev lost about one third of the records (33.8% of CVs and 31.6% of vacancies), because most of the non-developer roles were there. After cleaning, GameDev still has about 1,200 CVs and 170 vacancies.

Figure 4.3: Part of the records removed by cleaning in each role family



4.3 Data Normalization

Data normalization means preparing the data for the search methods. This work uses two methods, BM25 and Sentence-BERT, so there are two versions of the cleaned text.

Text for BM25. BM25 [6] looks for the same words in a CV and a vacancy. For it, `"Python"`, `"python"` and `"python,"` have to be the same word, and common words like *the* or *and* do not help to find a good match. Therefore the text for BM25 is prepared with the function `bm25_tokens` (Listing 4.2):

- All letters are made lowercase.
- The text is split into words and punctuation is removed. Names of technologies like `c++`, `c#`, `.net` and `node.js` stay one word.
- Common words (stop words) are removed using the English stop word list from the scikit-learn library [7].
- The placeholders `[LINK]`, `[EMAIL]` and `[PHONE]` are removed, because they do not say anything about skills.

Listing 4.2: Function for BM25 text normalization

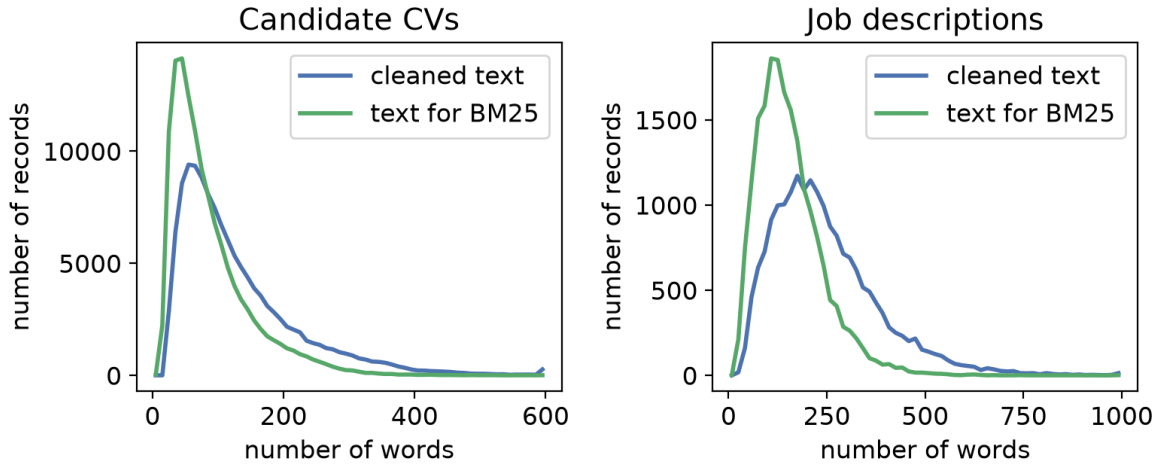
```
TOKEN = r"\.?[a-z0-9][a-z0-9+#]*(?:[.\-][a-z0-9+#]+)*"
PLACEHOLDERS = r"\[(?:link|email|phone)\]"

def bm25_tokens(text):
    text = re.sub(PLACEHOLDERS, " ", text.lower())
    return [w for w in re.findall(TOKEN, text) if w not in
        ENGLISH_STOP_WORDS]
```

For example, the sentence *"Experienced Python developer, working with Django, Node.js, C++ and .NET for 5 years."* becomes the following list of words: `experienced`, `python`, `developer`, `working`, `django`, `node.js`, `c++`, `.net`, `5`, `years`.

The original text is not changed. The result is saved in new columns `CV_bm25` and `Long Description_bm25`. After normalization texts became about one third shorter: the median is 68 words for a CV instead of 103, and 140 words for a vacancy instead of 220 (Figure 4.4).

Figure 4.4: Number of words before and after normalization for BM25



Text for Sentence-BERT. Sentence-BERT [8] understands the meaning of a whole sentence, so it gets the cleaned text from Section 4.2 without lowercase or removing stop words. Words like *not* or *without* are stop words, but they can change the meaning of a sentence.

Sentence-BERT turns every text into a vector. These vectors are normalized to length 1 (L2 normalization):

$$\hat{v} = \frac{v}{\|v\|_2}, \qquad \|v\|_2 = \sqrt{\sum_{i=1}^{n} v_i^2} \tag{4.1}$$

where v is the vector of a text and n is the number of values in it. After this, the cosine similarity between a CV and a vacancy can be calculated as a simple dot product of their vectors:

$$\cos(a, b) = \frac{a \cdot b}{\|a\|_2 \, \|b\|_2} = \hat{a} \cdot \hat{b} \tag{4.2}$$

where a is the vector of a CV and b is the vector of a vacancy. The vectors themselves are calculated when the search is done (Chapter 6).

4.4 Summary

After cleaning, there are no phone numbers, links, special characters or Cyrillic letters left, and non-developer roles were removed, so the labels are more correct. 126,734 CVs and 19,306 vacancies were left, which is about 96% of the data. Each record has two versions of the text: one for Sentence-BERT and one for BM25.

All steps can be repeated with the notebook `notebooks/02_data_cleaning.ipynb` (`random_state = 5267`). The following versions were used: Python 3.14.5, pandas 3.1.0, NumPy 2.5.4, Matplotlib 3.11.2, scikit-learn 1.9.1 and ftfy 6.3.1.

References

1. N. Drushchak and M. Romanyshyn, *"Introducing the Djinni Recruitment Dataset: A Corpus of Anonymized CVs and Job Postings,"* in Proceedings of the Third Ukrainian Natural Language Processing Workshop (UNLP) @ LREC-COLING 2024, Torino, Italy, pp. 8–13, 2024. URL: https://aclanthology.org/2024.unlp-1.2

2. W. McKinney, *"Data Structures for Statistical Computing in Python,"* in Proceedings of the 9th Python in Science Conference (SciPy 2010), Austin, TX, USA, pp. 56–61, 2010. URL: https://doi.org/10.25080/Majora-92bf1922-00a

3. J. D. Hunter, *"Matplotlib: A 2D Graphics Environment,"* Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. URL: https://doi.org/10.1109/MCSE.2007.55

4. R. Speer, *"ftfy: fixes text for you,"* Python library, version 6.3.1. URL: https://github.com/rspeer/python-ftfy

5. The Unicode Consortium, *"Unicode Standard Annex #15: Unicode Normalization Forms."* URL: https://unicode.org/reports/tr15/

6. S. Robertson and H. Zaragoza, *"The Probabilistic Relevance Framework: BM25 and Beyond,"* Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009. URL: https://doi.org/10.1561/1500000019

7. F. Pedregosa et al., *"Scikit-learn: Machine Learning in Python,"* Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. URL: https://jmlr.org/papers/v12/pedregosa11a.html

8. N. Reimers and I. Gurevych, *"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,"* in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, pp. 3982–3992, 2019. URL: https://doi.org/10.18653/v1/D19-1410